



NoSQL Databases

Redis · Cassandra · MongoDB · Neo4j · DynamoDB

Overview

Polyglot persistence: Redis (cache+messaging), Cassandra (event log), DynamoDB (sessions), MongoDB (documents), Neo4j (graph), InfluxDB/Prometheus (time-series). Each store chosen for its CAP properties and data model fit.

Data Store Decision Matrix

Store	CAP	Data Model	ShopVerse Use Case
PostgreSQL	CP	Relational	Orders, Customers, Products (source of truth)
Redis	CP (cluster)	Key-Value / Data Structure	Cart, Session, Rate Limit, Pub/Sub, Streams
Cassandra	AP	Wide-Column	OrderEventLog (append-only, time-series)
MongoDB	CP	Document	Product specs, Reviews (flexible schema)
Neo4j	CA	Graph	Product recommendations (collaborative filtering)
DynamoDB	AP	Key-Value	UserSession (AWS; TTL, GSI on email)
Elasticsearch	AP	Inverted Index	Full-text product search, autocomplete
InfluxDB	CP	Time-Series	Business metrics (orders/min, revenue/hr)

Redis Data Structures

Structure	Class	Command Pattern
String	SessionTokenStore	SET session:{token} {userId} EX 3600
Hash	ShoppingCartService	HSET cart:{cid} {pid} {qty} / HGETALL / EXPIRE
List	RecentlyViewedService	LPUSH viewed:{cid} {pid} + LTRIM 0 9 (last 10)
Set	ProductTagIndex	SADD tag:{name} {pid} / SINTER tag:electronics tag:sale
Sorted Set	ProductLeaderboard	ZADD leaderboard {score} {pid} / ZREVRANGE 0 9
Pub/Sub	InventoryPubSub	PUBLISH inventory:low / LowStockSubscriber
Lua Script	AtomicRateLimiter	Atomic INCR+EXPIRE (no race condition)
Streams	OrderEventStream	XADD orders:events * orderId ... / XREAD consumer group

Cassandra Schema (OrderEventLog)

```
PRIMARY KEY (order_id, event_time) -- partition: order_id, cluster: event_time DESC
Consistency: QUORUM for writes / LOCAL_ONE for reads
```

Keyspace: SimpleStrategy (dev, RF=1) → NetworkTopologyStrategy (prod).

MongoDB Aggregation Pipeline

```
Aggregation.newAggregation(
  match(where('productId').is(id)),
  group('productId').avg('rating').as('avgRating'),
  sort(DESC, 'avgRating'))
```

Text index: @TextIndexed on ProductDocument.name + description → TextCriteria for full-text.

Neo4j Graph Model

(:Customer) – id, email

–[:PURCHASED {orderId, purchasedAt, quantity}]→ (:Product)

RecommendationRepository Cypher:

MATCH (c)-[:PURCHASED]->(p)<-[:PURCHASED]-(other)-[:PURCHASE

WHERE NOT (c)-[:PURCHASED]->(rec)

RETURN rec, COUNT(*) AS score ORDER BY score DESC LIMIT 10